

НИУ ИТМО
Факультет ПИиКТ

Лабораторная работа №3
Численное интегрирование
Вариант №5

Преподаватель **Наумова Надежда Александровна**
Подготовил **Лоскутов Прохор Александрович**

Санкт - Петербург 2026

Оглавление

Оглавление	2
1 Вычислительная реализация задачи	3
1.1 Точное значение по формуле Ньютона–Лейбница	3
1.2 Формула Ньютона–Котеса при $n = 6$	3
1.3 Численные методы при $n = 10$	4
2 Программная реализация задачи	6
2.1 Рабочие формулы и блок-схемы реализованных методов .	7
2.2 Исходный код основных элементов расчёта	20
3 Выводы	31

1 Вычислительная реализация задачи

По варианту №5 необходимо вычислить определённый интеграл:

$$I = \int_2^4 (-2x^3 - 3x^2 + x + 5)dx$$

Согласно заданию, интеграл вычисляется четырьмя способами: точно (по формуле Ньютона–Лейбница), по общей формуле Ньютона–Котеса при $n = 6$ и по составным квадратурным формулам средних прямоугольников, трапеций и Симпсона при $n = 10$. Полученные значения сравниваются с точным.

1.1 Точное значение по формуле Ньютона–Лейбница

Первообразная:

$$F(x) = \int (-2x^3 - 3x^2 + x + 5)dx = -\frac{x^4}{2} - x^3 + \frac{x^2}{2} + 5x$$

$$I = -164 - (-4) = -160$$

1.2 Формула Ньютона–Котеса при $n = 6$

Формула Ньютона–Котеса порядка n строится как интеграл от полинома Лагранжа $L_n(x)$, проходящего через $n + 1$ равноотстоящий узел $x_i = a + ih$, $h = (b - a)/n$:

$$\int_a^b f(x)dx \approx \sum_{i=0}^n c_n^i \cdot f(x_i)$$

где c_n^i — коэффициенты Котеса. Для $n = 6$ из таблицы лекции:

$$c_6^0 = c_6^6 = \frac{41(b-a)}{840}, \quad c_6^1 = c_6^5 = \frac{216(b-a)}{840},$$

$$c_6^2 = c_6^4 = \frac{27(b-a)}{840}, \quad c_6^3 = \frac{272(b-a)}{840}$$

Шаг: $h = \frac{b-a}{n} = 1/3$, узлы: $x_i = 2 + i/3$, $i = 0, \dots, 6$.

Таблица 1

Значения функции в узлах формулы Ньютона–Котеса

i	x_i	$f(x_i)$
0	2	-21.0000
1	7/3	-34.4074
2	8/3	-51.5926
3	3	-73.0000
4	10/3	-99.0741
5	11/3	-130.2593
6	4	-167.0000

Вынося общий множитель $(b - a)/840 = 2/840$, формула принимает вид:

$$I_{\text{HK6}} = \frac{b - a}{840} \cdot (41f(x_0) + 216f(x_1) + 27f(x_2) + 272f(x_3) + 27f(x_4) + 216f(x_5) + 41f(x_6))$$

Подставляя значения из таблицы 1 и сразу собирая взвешенную сумму:

$$\begin{aligned} I_{\text{HK6}} &= \frac{2}{840} \cdot (-861 - 7432 - 1393 - 19856 \\ &\quad - 2675 - 28136 - 6847) \\ &= \frac{2}{840} \cdot (-67200) = -160 \end{aligned}$$

Результат совпадает с точным значением.

1.3 Численные методы при $n = 10$

Шаг разбиения: $h = \frac{b-a}{n} = \frac{2}{10} = 0.2$.

Таблица 2

Значения функции в узловых и полуцелых точках

i	x_i	y_i	$x_{i-1/2}$	$y_{i-1/2}$
0	2.0	-21.0000	—	—
1	2.2	-28.6160	2.1	-24.6520
2	2.4	-37.5280	2.3	-32.9040
3	2.6	-47.8320	2.5	-42.5000
4	2.8	-59.6240	2.7	-53.5360
5	3.0	-73.0000	2.9	-66.1080

6	3.2	−88.0560		3.1	−80.3120
7	3.4	−104.8880		3.3	−96.2440
8	3.6	−123.5920		3.5	−114.0000
9	3.8	−144.2640		3.7	−133.6760
10	4.0	−167.0000		3.9	−155.3680

1.3.1 Метод средних прямоугольников

$$I_{\text{сред}} = h \cdot \sum_{i=1}^{10} y_{i-1/2}$$

Сумма значений в полуцелых точках:

$$\begin{aligned} S_{\text{сред}} &= -24.6520 - 32.9040 - 42.5000 - 53.5360 \\ &\quad - 66.1080 - 80.3120 - 96.2440 - 114.0000 \\ &\quad - 133.6760 - 155.3680 = -799.3000 \\ I_{\text{сред}} &= 0.2 \cdot (-799.3000) = -\mathbf{159.8600} \end{aligned}$$

1.3.2 Метод трапеций

$$\begin{aligned} I_{\text{трап}} &= \frac{h}{2} \cdot \left(y_0 + y_{10} + 2 \sum_{i=1}^9 y_i \right) \\ \sum_{i=1}^9 y_i &= -28.6160 - 37.5280 - 47.8320 - 59.6240 \\ &\quad - 73.0000 - 88.0560 - 104.8880 - 123.5920 \\ &\quad - 144.2640 = -707.4000 \\ I_{\text{трап}} &= 0.1 \cdot (-21.0000 - 167.0000 + 2 \cdot (-707.4000)) \\ &= 0.1 \cdot (-1602.8000) = -\mathbf{160.2800} \end{aligned}$$

1.3.3 Метод Симпсона

$$I_{\text{Симп}} = \frac{h}{3} \cdot \left(y_0 + y_{10} + 4 \sum_{\text{нечёт}} y_i + 2 \sum_{\text{чёт} > 0} y_i \right)$$

Нечётные узлы ($i = 1, 3, 5, 7, 9$):

$$\begin{aligned} S_{\text{нечёт}} &= -28.6160 - 47.8320 - 73.0000 \\ &\quad - 104.8880 - 144.2640 = -398.6000 \end{aligned}$$

Чётные внутренние ($i = 2, 4, 6, 8$):

$$S_{\text{чёт}} = -37.5280 - 59.6240 - 88.0560 - 123.5920 = -308.8000$$

$$\begin{aligned} I_{\text{Симп}} &= \frac{0.2}{3} \cdot (-21 - 167 + 4 \cdot (-398.6000) + 2 \cdot (-308.8000)) \\ &= \frac{1}{15} \cdot (-2400) = -160.0000 \end{aligned}$$

1.3.4 Сравнение с точным значением

Таблица 3

Сравнение методов с точным значением $I = -160$

Метод	Значение	$\Delta = I - I_{\text{точн}} $	δ (отн.), %
Ньютон–Котес ($n = 6$)	-160.0000	0.0000	0.0000
Средние прямоуголь- ники ($n = 10$)	-159.8600	0.1400	0.0875
Трапеции ($n = 10$)	-160.2800	0.2800	0.1750
Симпсон ($n = 10$)	-160.0000	0.0000	0.0000

Формула Ньютона–Котеса при $n = 6$ и Симпсон при $n = 10$ дают точное значение. Средние прямоугольники вдвое точнее трапеций — это согласуется с теоретическими коэффициентами оценки погрешности ($1/24$ против $1/12$).

2 Программная реализация задачи

Согласно заданию, в программе реализованы:

Метод / алгоритм	Назначение
Левые прямоугольники	Квадратурная формула
Правые прямоугольники	Квадратурная формула
Средние прямоугольники	Квадратурная формула
Трапеции	Квадратурная формула
Симпсон	Квадратурная формула
Правило Рунге	Оценка погрешности и автовыбор n
Обработка несобственных интегралов 2-го рода	Необязательное задание

Главное значение Коши (V.p.)	Расширение для антисимметричных особенностей
------------------------------	--

Каждый метод оформлен как отдельный класс-наследник `Integrator` — по аналогии с базовым `solver` из ЛР №2. Базовый класс хранит точность `EPS` и реализует общие алгоритмы (правило Рунге, обработка несобственных интегралов), а наследники переопределяют только виртуальный метод `apply(f, a, b, n)` — собственно квадратурную формулу.

2.1 Рабочие формулы и блок-схемы реализованных методов

2.1.1 Метод левых прямоугольников

Отрезок $[a, b]$ разбивается на n равных частей с шагом $h = (b - a)/n$. На каждом частичном отрезке функция заменяется константой — значением в его левой границе.

Рабочая формула:

$$I_{\text{лев}} = h \cdot \sum_{i=0}^{n-1} f(a + ih)$$

Порядок точности: $O(h)$.

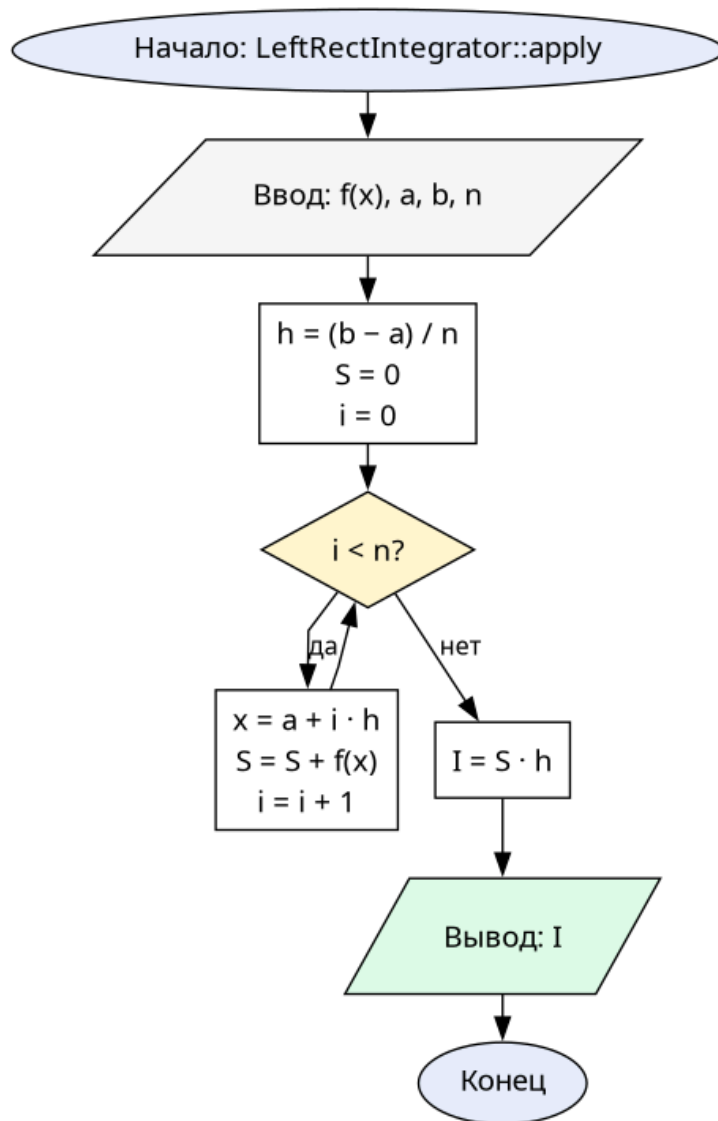


Рис. 1. Блок-схема метода левых прямоугольников

2.1.2 Метод правых прямоугольников

Аналогично левым, но представителем выбирается правая граница частичного отрезка.

Рабочая формула:

$$I_{\text{прав}} = h \cdot \sum_{i=1}^n f(a + ih)$$

Порядок точности: $O(h)$.

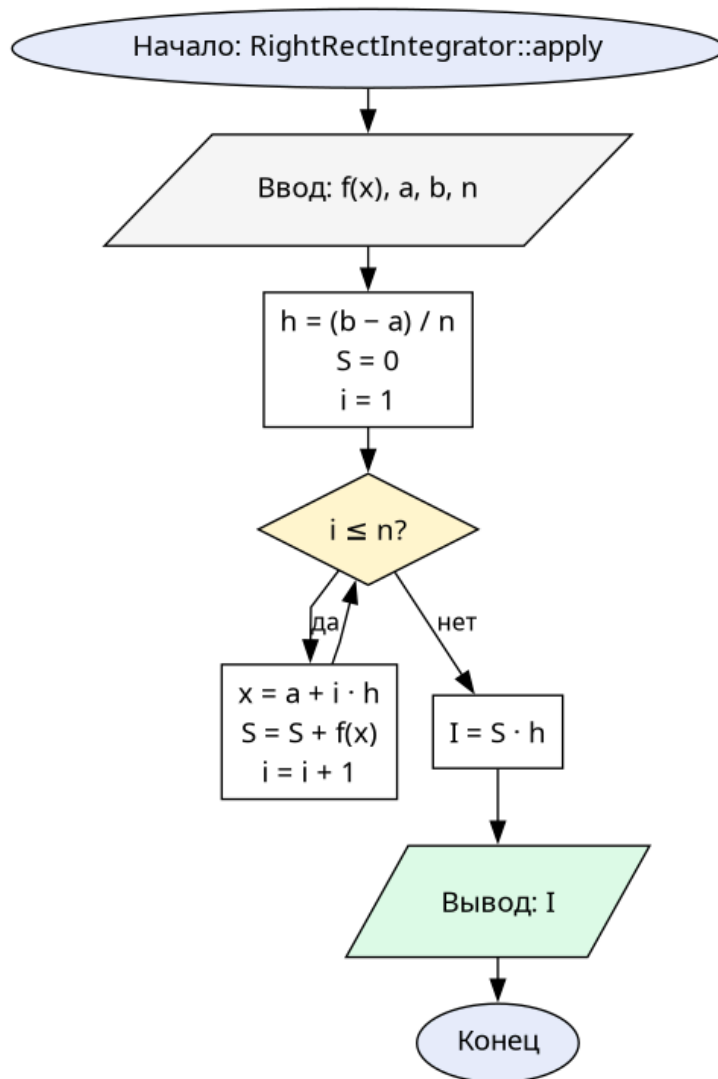


Рис. 2. Блок-схема метода правых прямоугольников

2.1.3 Метод средних прямоугольников

Представителем выбирается середина частичного отрезка — за счёт симметрии вокруг середины часть погрешности линейного приближения сокращается, поэтому метод заметно точнее левых и правых прямоугольников.

Рабочая формула:

$$I_{\text{сред}} = h \cdot \sum_{i=1}^n f\left(a + \left(i - \frac{1}{2}\right)h\right)$$

Порядок точности: $O(h^2)$.

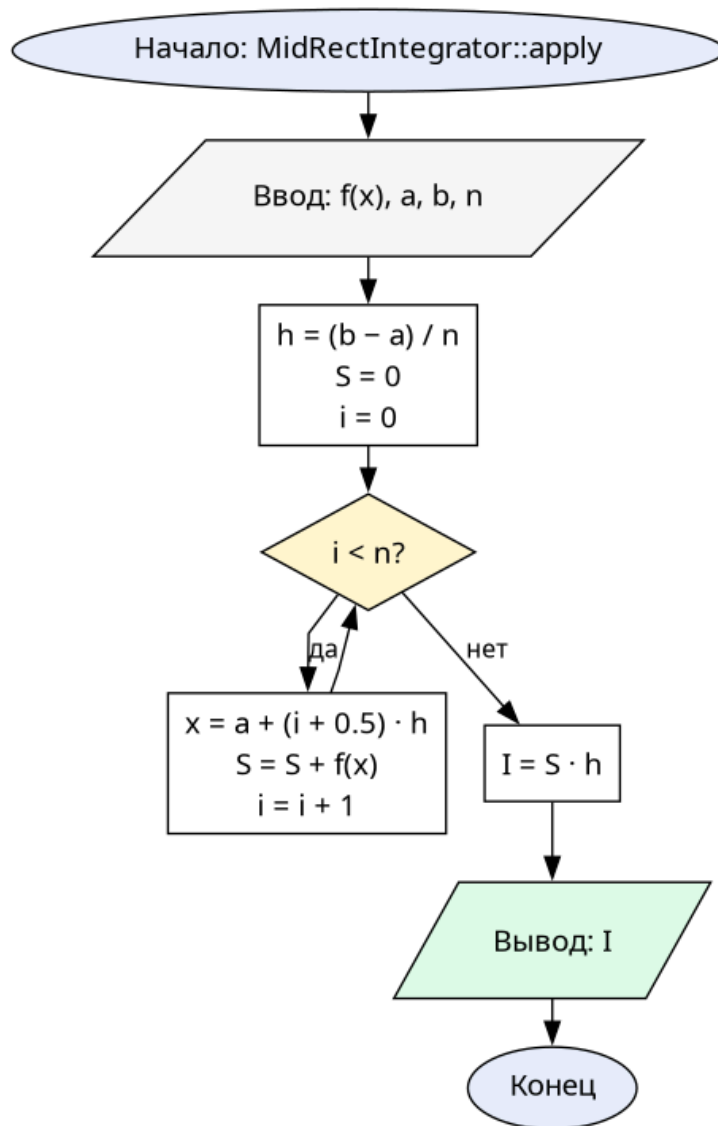


Рис. 3. Блок-схема метода средних прямоугольников

2.1.4 Метод трапеций

На каждом частичном отрезке функция заменяется линейным полиномом, проходящим через два соседних узла; интеграл от линейного полинома — площадь трапеции.

Рабочая формула:

$$I_{\text{трап}} = \frac{h}{2} \cdot \left(y_0 + y_n + 2 \sum_{i=1}^{n-1} y_i \right), \quad y_i = f(a + ih)$$

Порядок точности: $O(h^2)$.

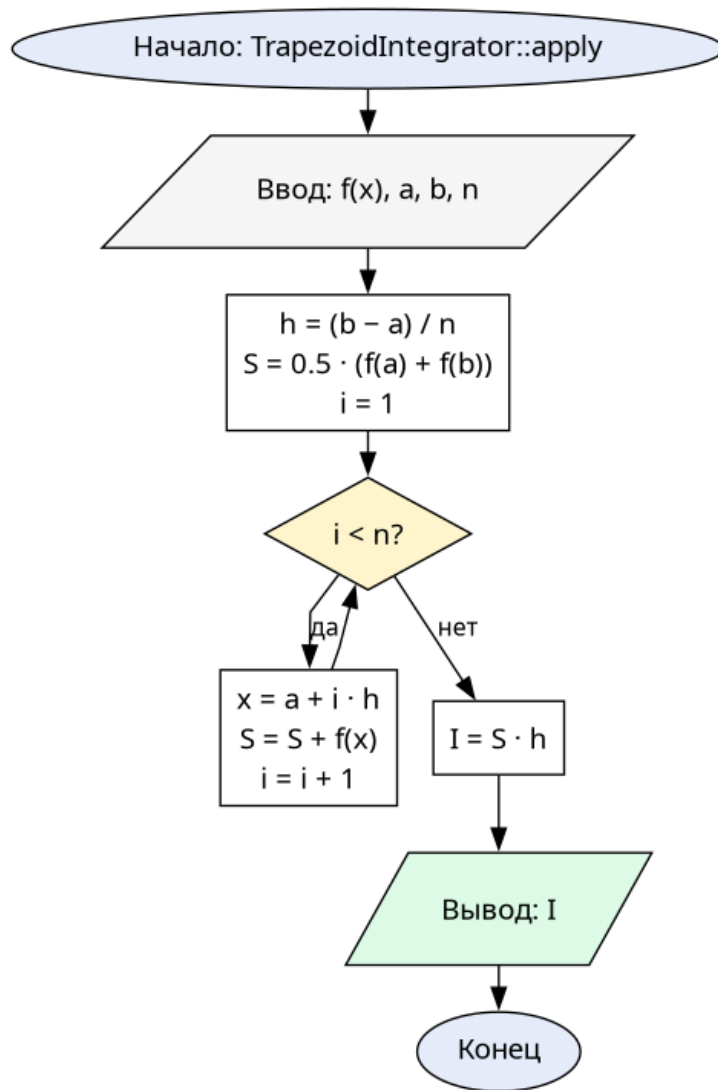


Рис. 4. Блок-схема метода трапеций

2.1.5 Метод Симпсона

На каждой паре частичных отрезков $[x_{i-1}, x_{i+1}]$ функция аппроксимируется квадратичным многочленом Лагранжа. Требуется чётное n .

Рабочая формула:

$$I_{\text{Симп}} = \frac{h}{3} \cdot \left(y_0 + y_n + 4 \sum_{\text{нечёт}} y_i + 2 \sum_{\text{чёт} > 0} y_i \right)$$

Порядок точности: $O(h^4)$. На полиномах степени не выше трёх метод даёт точный результат.

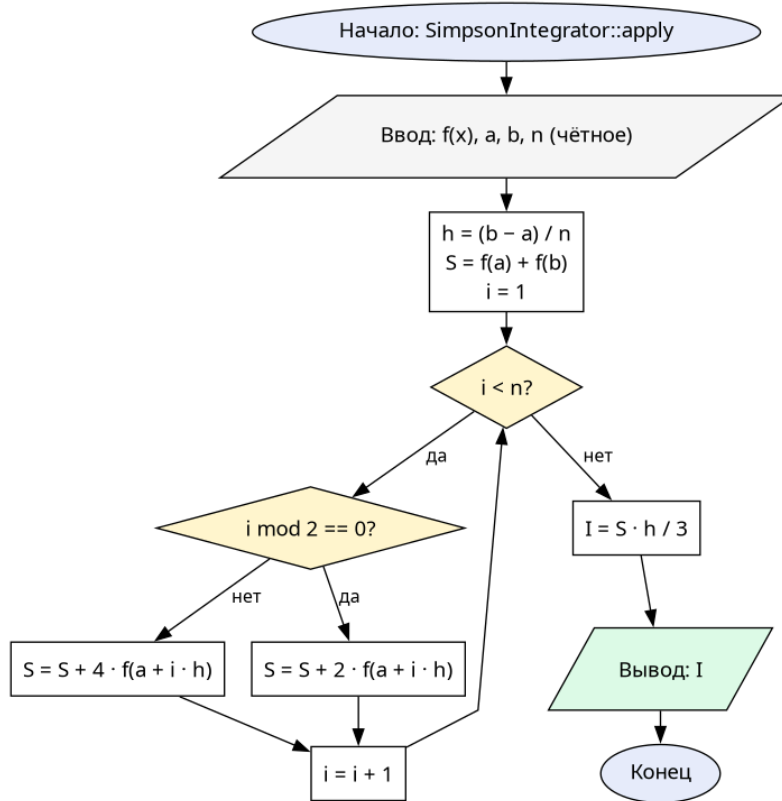


Рис. 5. Блок-схема метода Симпсона

2.1.6 Правило Рунге

Эмпирическая оценка погрешности квадратурной формулы порядка k через результаты с шагами h и $h/2$:

$$R \approx \frac{|I_{h/2} - I_h|}{2^k - 1}$$

где $k = 2$ для прямоугольников и трапеций, $k = 4$ для Симпсона. Алгоритм автовыбора n : начиная с $n = 4$, последовательно удваиваем n и пересчитываем интеграл $I_{h/2}$. Если $R \leq \varepsilon$, вычисление останавливается и $I_{h/2}$ принимается в качестве приближения. Критерий окончания: $R \leq \varepsilon$.

Защита от зависания: лимит $n_{\max} = 2^{20}$.

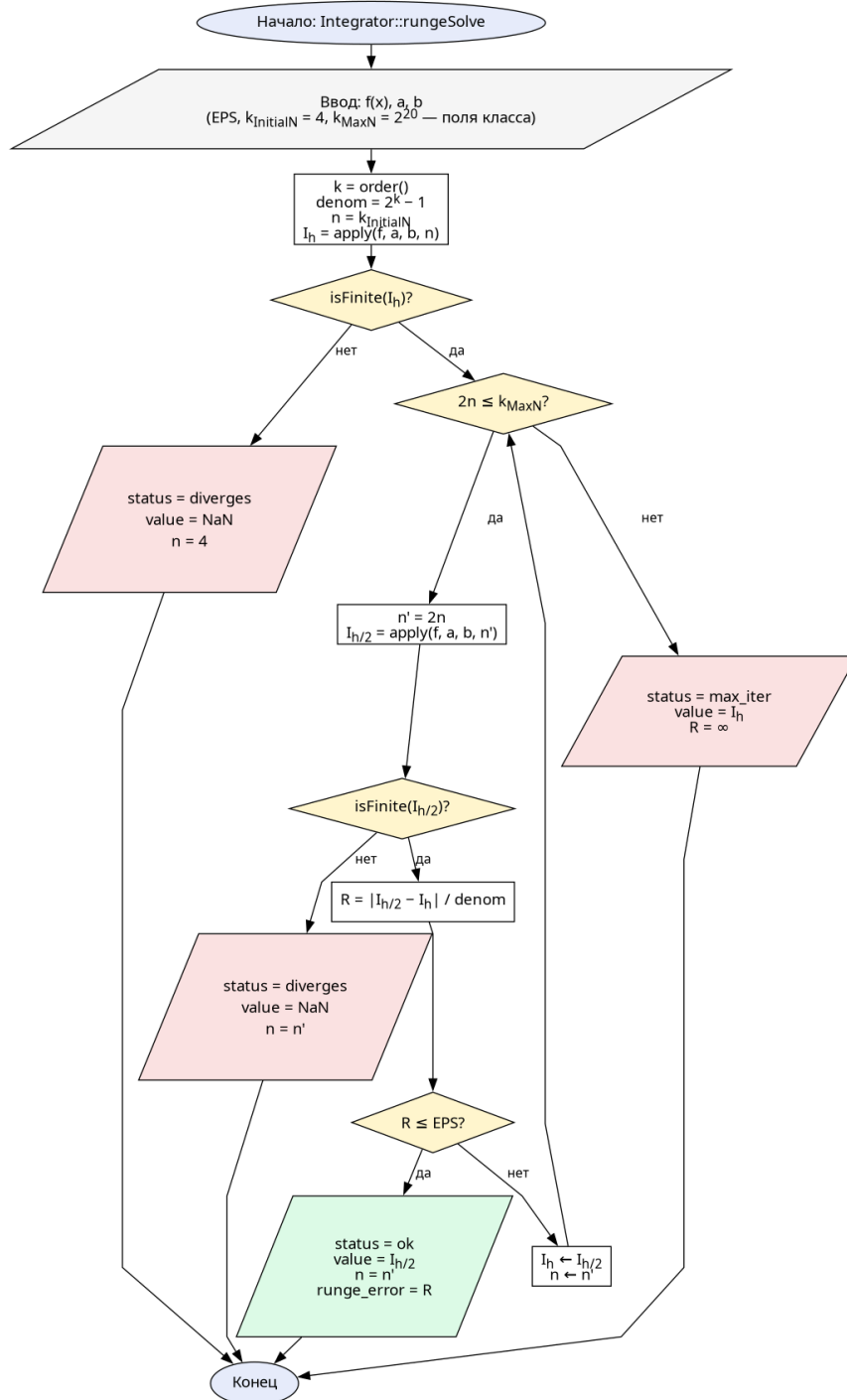


Рис. 6. Алгоритм удвоения n по правилу Рунге

2.1.7 Обработка несобственных интегралов 2-го рода

Если подынтегральная функция терпит бесконечный разрыв в точке $c \in [a, b]$, интеграл определяется через предельный переход. Для разрыва на левой границе ($c = a$):

$$\int_a^b f(x)dx = \lim_{\sigma \rightarrow +0} \int_{a+\sigma}^b f(x)dx$$

Аналогично для правой границы и для внутреннего разрыва (сумма двух односторонних пределов).

Алгоритм собран из пяти взаимодействующих процедур, каждая из которых отражена отдельной блок-схемой:

- `prepareIntervals` — нарезает $[a, b]$ по точкам разрыва, расставляет флаги `limit_at_from` / `limit_at_to`;
- `computeSegmentStandard` — для каждого сегмента выбирает обычный путь (`rungeSolve`) или вычисление одностороннего предела (`limitAtEndpoint`);
- `limitAtEndpoint` — итеративное сужение к точке разрыва с проверкой стабилизации;
- `tryPrincipalValue` — fallback на главное значение Коши, если стандартный путь не сошёлся;
- `isAntisymmetric` — численная проверка нечётности функции вокруг точки разрыва, используется в `tryPrincipalValue`.

Верхняя процедура `Integrator::integrate` оркестрирует эти подпрограммы.

Критерий сходимости предела (внутри `limitAtEndpoint`): $|I_k - I_{k-1}| < 3\epsilon$ для двух подряд k , причём внутренний `rungeSolve` действительно сошёлся (статус `ok`, не `max_iter`).

Критерии расходимости: либо `apply` вернул NaN, либо $|I_k| > 10^{15}$ на каком-то шаге.

Критерий «не определилось»: последовательность не стабилизировалась за $k_{\max} = 50$ сужений (это не расходимость, а ограничение метода).

Главное значение Коши. Для внутреннего разрыва c , если стандартное вычисление вернуло не-`ok`, проверяется численная антисимметричность f в окрестности c на подотрезке $[c - d, c + d]$, $d = \min(c - a, b - c)$. Если максимум относительной разницы $|f(c + \delta) + f(c - \delta)| / \max(|f(c + \delta)|, |f(c - \delta)|)$ по 10 пробным точкам δ меньше 10^{-6} , подотрезок $[c - d, c + d]$ зануляется (в смысле V.p.), а оставшиеся «хвосты» интегрируются как обычно. Результат помечается флагом `ok_principal_value`.

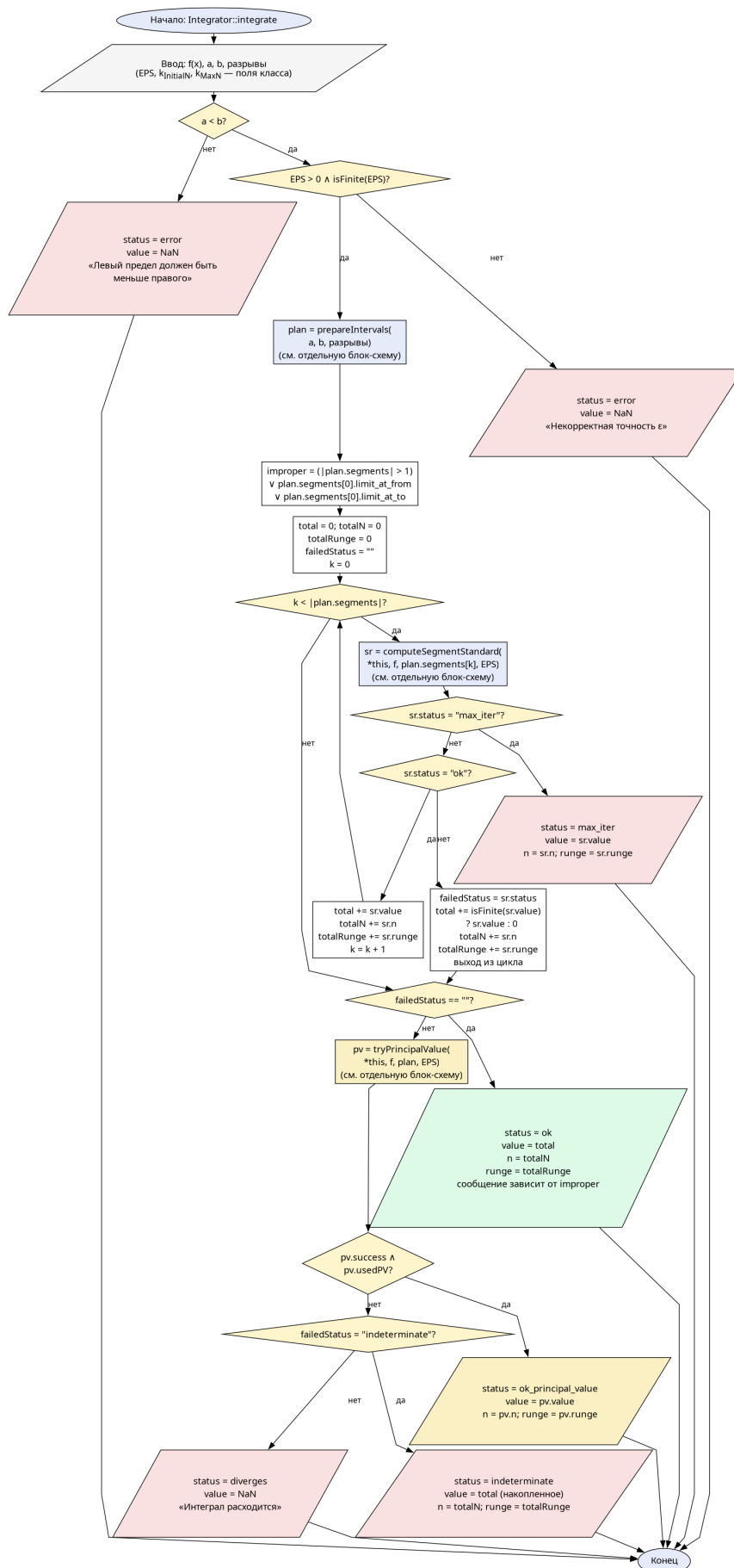


Рис. 7. Верхнеуровневая схема Integrator::integrate (подпрограммы — отдельные блок-схемы)

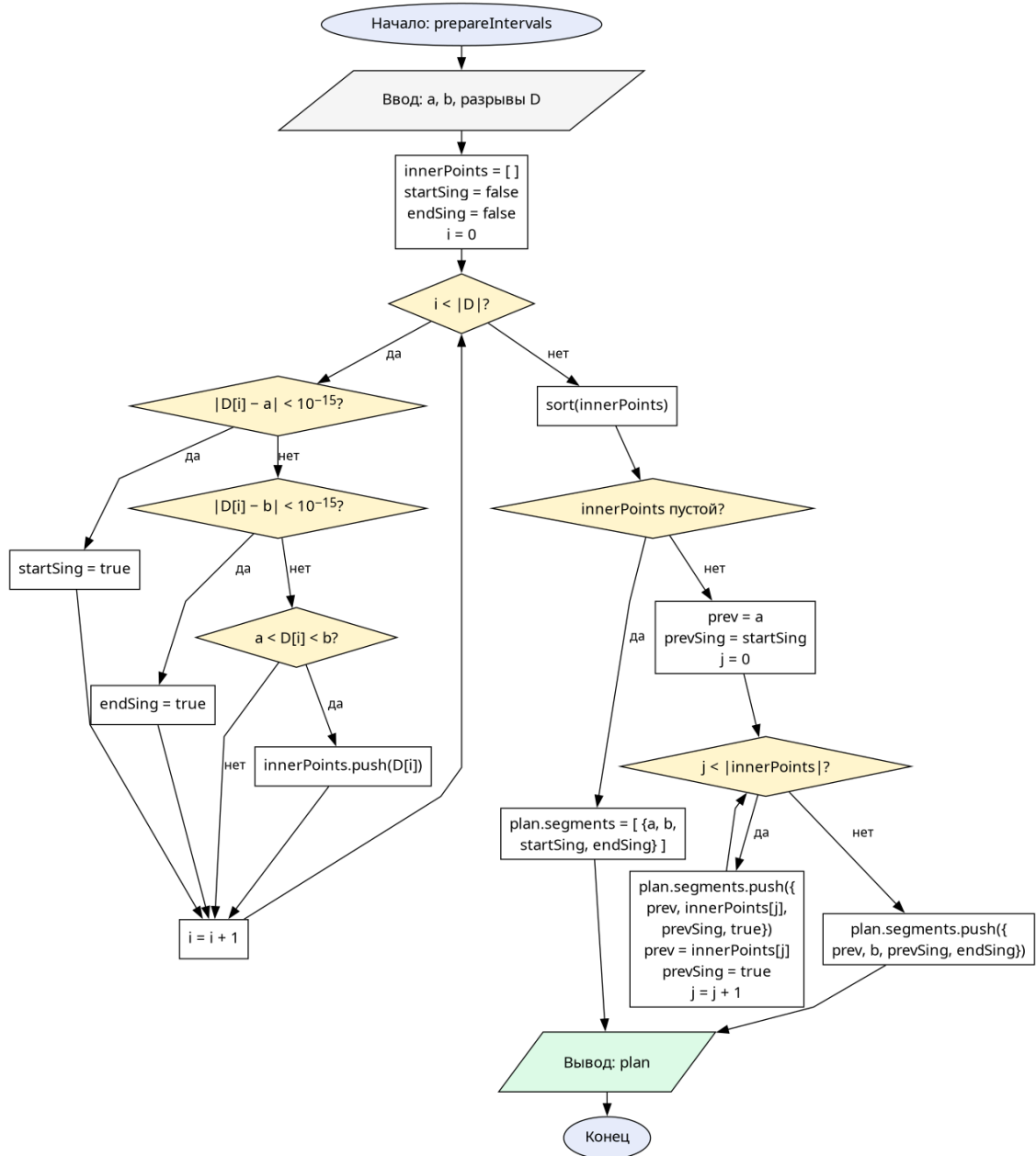


Рис. 8. prepareIntervals — декомпозиция отрезка по точкам разрыва

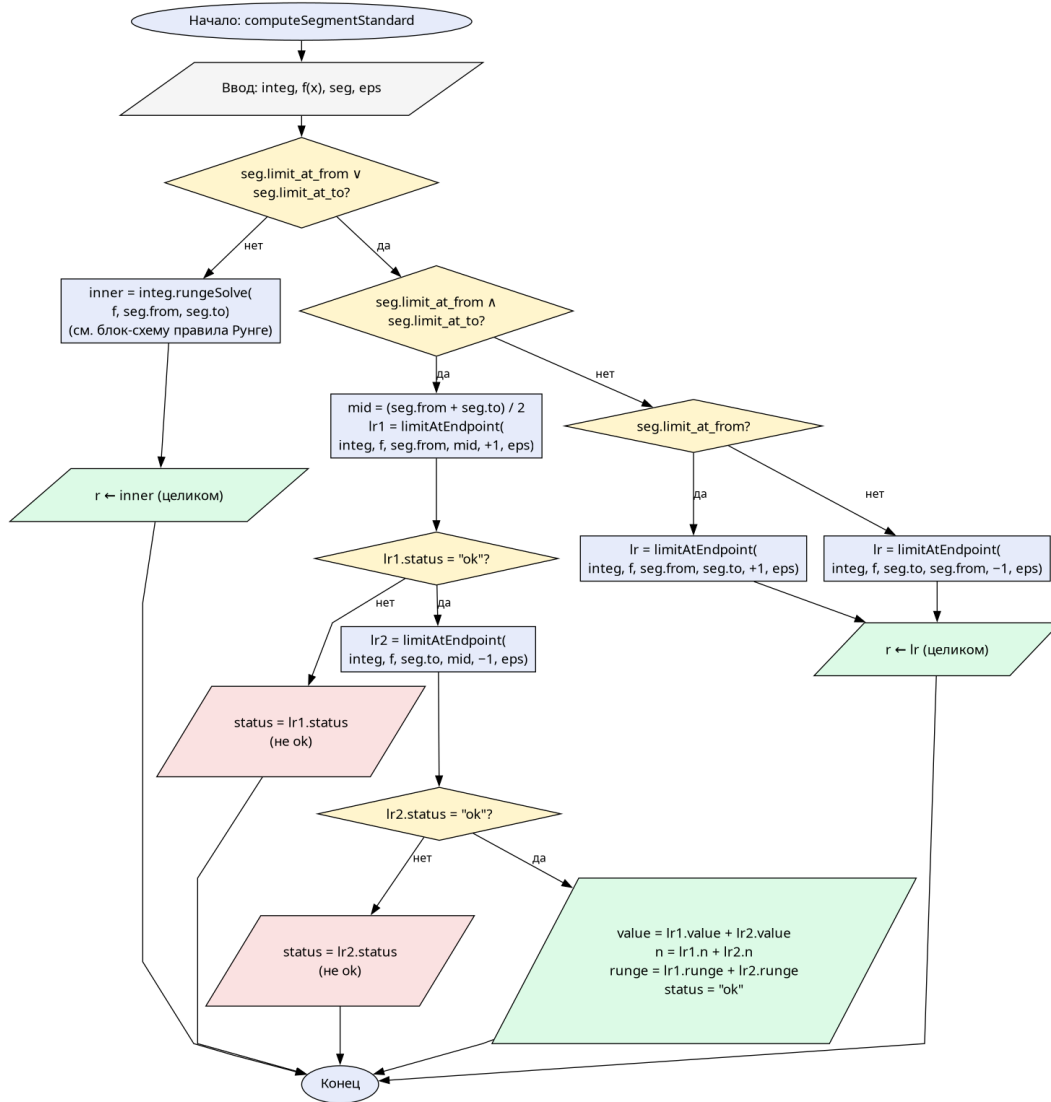


Рис. 9. computeSegmentStandard — выбор пути для одного сегмента

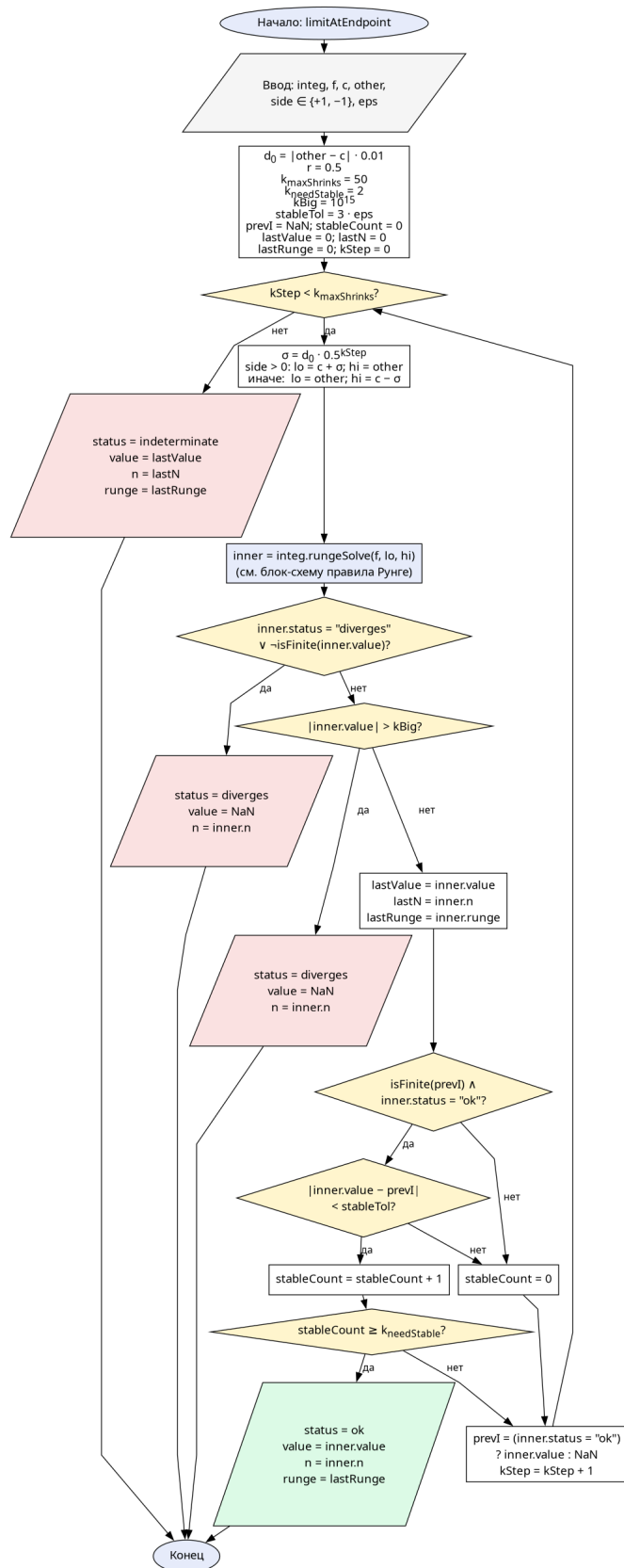


Рис. 10. `limitAtEndpoint` — численный односторонний предел через сужение

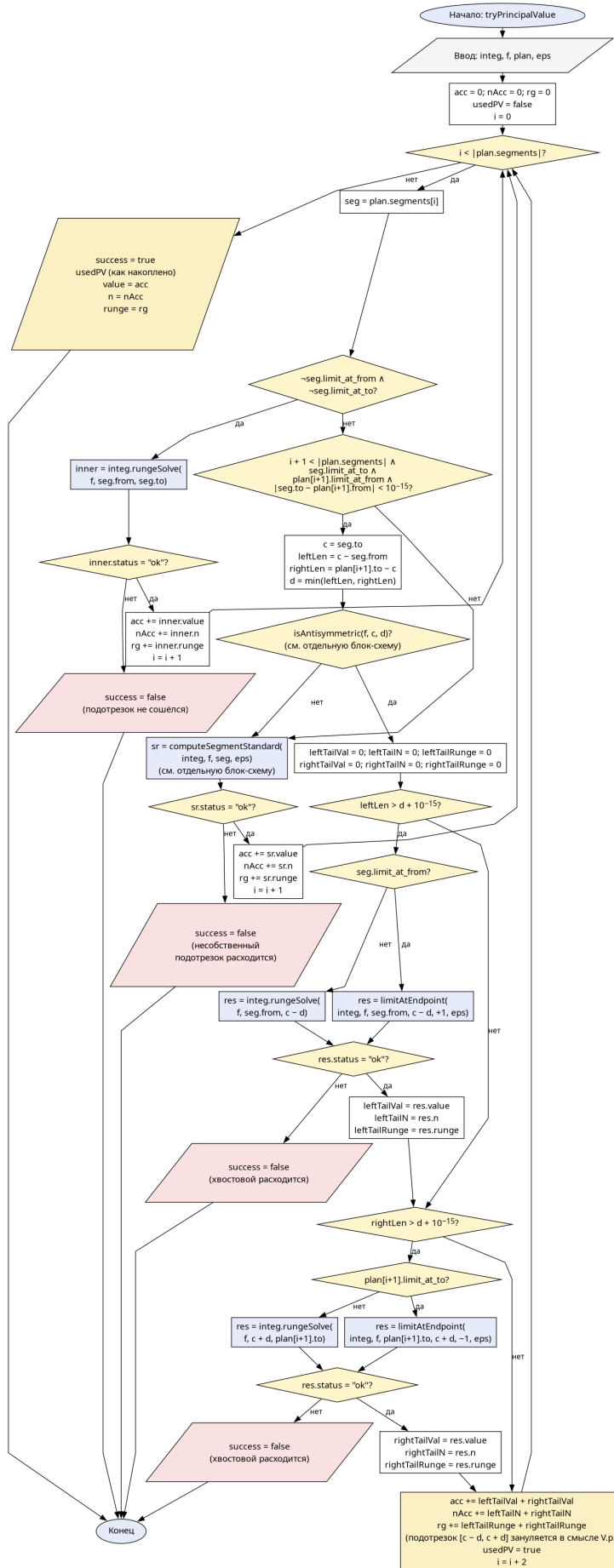


Рис. 11. tryPrincipalValue — fallback на главное значение Коши

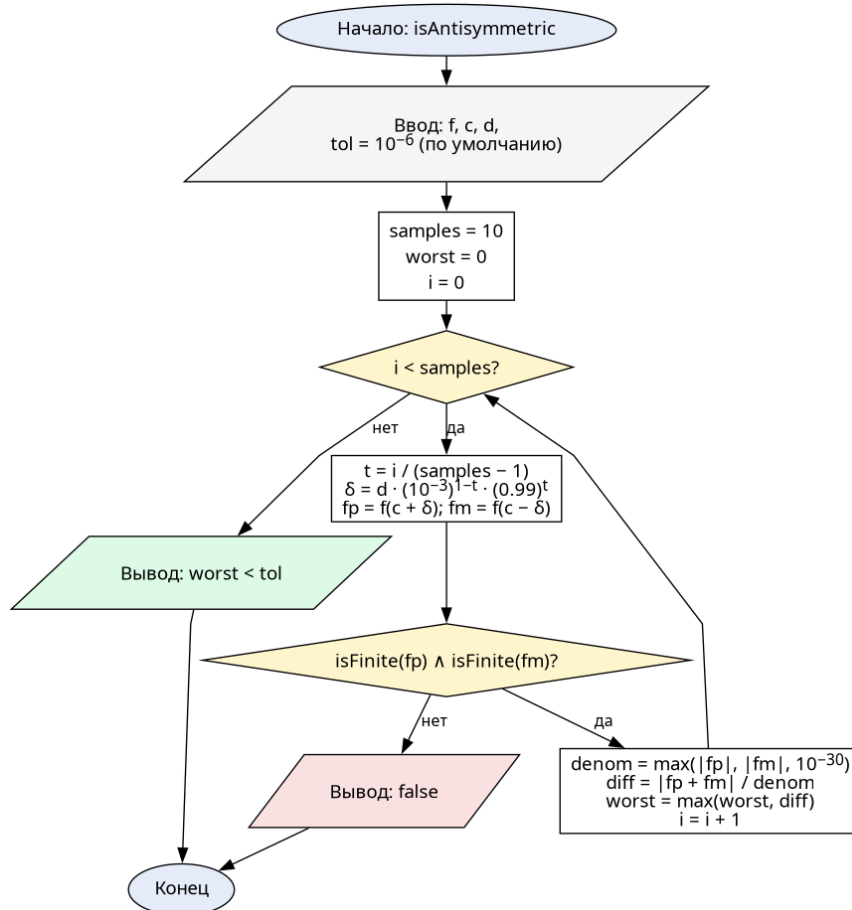


Рис. 12. `isAntisymmetric` — численная проверка нечётности f вокруг точки c

2.2 Исходный код основных элементов расчёта

Заголовочный файл `calc/Integrators.hpp` с базовым классом `Integrator` (правило Рунге и обработка несобственных интегралов) и пятью наследниками-квадратурами:

```

#pragma once

#include <functional>
#include <string>
#include <vector>

using IntegrandFunc = std::function<double(double)>;

struct IntegrationResult {
    double value = 0.0;
    int n = 0;
    double runge_error = 0.0;
    std::string status;
    std::string message;
};

class Integrator {
protected:
    double EPS;

```

```

public:
    static constexpr int kInitialN = 4;
    static constexpr int kMaxN = 1 << 20;

    Integrator(double EPS) : EPS(EPS) {};
    virtual ~Integrator() = default;

    virtual double apply(IntegrandFunc f, double a, double b, int n) = 0;
    virtual int order() = 0;

    IntegrationResult rungeSolve(IntegrandFunc f, double a, double b);
    IntegrationResult integrate(IntegrandFunc f, double a, double b,
                                const std::vector<double> &discontinuities);
};

class LeftRectIntegrator : public Integrator {
public:
    LeftRectIntegrator(double EPS) : Integrator(EPS) {};
    double apply(IntegrandFunc f, double a, double b, int n) override;
    int order() override { return 2; }
};

class RightRectIntegrator : public Integrator {
public:
    RightRectIntegrator(double EPS) : Integrator(EPS) {};
    double apply(IntegrandFunc f, double a, double b, int n) override;
    int order() override { return 2; }
};

class MidRectIntegrator : public Integrator {
public:
    MidRectIntegrator(double EPS) : Integrator(EPS) {};
    double apply(IntegrandFunc f, double a, double b, int n) override;
    int order() override { return 2; }
};

class TrapezoidIntegrator : public Integrator {
public:
    TrapezoidIntegrator(double EPS) : Integrator(EPS) {};
    double apply(IntegrandFunc f, double a, double b, int n) override;
    int order() override { return 2; }
};

class SimpsonIntegrator : public Integrator {
public:
    SimpsonIntegrator(double EPS) : Integrator(EPS) {};
    double apply(IntegrandFunc f, double a, double b, int n) override;
    int order() override { return 4; }
};

```

Реализация (calc/integrators.cpp) — собственно квадратурные формулы, правило Рунге, разбиение по точкам разрыва, односторонние пределы и главное значение Коши:

```

#include "Integrators.hpp"

#include <algorithm>
#include <cmath>

```

```

#include <limits>

double LeftRectIntegrator::apply(IntegrandFunc f, double a, double b, int n) {
    const double h = (b - a) / static_cast<double>(n);
    double sum = 0.0;
    for (int i = 0; i < n; ++i) {
        const double x = a + i * h;
        sum += f(x);
    }
    return sum * h;
}

double RightRectIntegrator::apply(IntegrandFunc f, double a, double b, int n) {
    const double h = (b - a) / static_cast<double>(n);
    double sum = 0.0;
    for (int i = 1; i ≤ n; ++i) {
        const double x = a + i * h;
        sum += f(x);
    }
    return sum * h;
}

double MidRectIntegrator::apply(IntegrandFunc f, double a, double b, int n) {
    const double h = (b - a) / static_cast<double>(n);
    double sum = 0.0;
    for (int i = 0; i < n; ++i) {
        const double x = a + (i + 0.5) * h;
        sum += f(x);
    }
    return sum * h;
}

double TrapezoidIntegrator::apply(IntegrandFunc f, double a, double b, int n) {
    const double h = (b - a) / static_cast<double>(n);
    double sum = 0.5 * (f(a) + f(b));
    for (int i = 1; i < n; ++i) {
        const double x = a + i * h;
        sum += f(x);
    }
    return sum * h;
}

double SimpsonIntegrator::apply(IntegrandFunc f, double a, double b, int n) {
    const double h = (b - a) / static_cast<double>(n);
    double sum = f(a) + f(b);
    for (int i = 1; i < n; ++i) {
        const double x = a + i * h;
        sum += (i % 2 == 0 ? 2.0 : 4.0) * f(x);
    }
    return sum * h / 3.0;
}

IntegrationResult Integrator::rungeSolve(IntegrandFunc f, double a, double b) {
    IntegrationResult result;
    const int k = order();
    const double denom = std::pow(2.0, k) - 1.0;

    int n = kInitialN;
    double prev = apply(f, a, b, n);
    if (!std::isfinite(prev)) {

```

```

    result.status = "diverges";
    result.message = "Начальное значение интеграла не конечно.";
    result.value = std::numeric_limits<double>::quiet_NaN();
    result.n = n;
    return result;
}

while (n * 2 ≤ kMaxN) {
    const int nNew = n * 2;
    const double cur = apply(f, a, b, nNew);
    if (!std::isfinite(cur)) {
        result.status = "diverges";
        result.message = "Значение интеграла не конечно.";
        result.value = std::numeric_limits<double>::quiet_NaN();
        result.n = nNew;
        return result;
    }
    const double runge = std::abs(cur - prev) / denom;
    if (runge ≤ EPS) {
        result.status = "ok";
        result.value = cur;
        result.n = nNew;
        result.runge_error = runge;
        return result;
    }
    prev = cur;
    n = nNew;
}

result.status = "max_iter";
result.message = "Достигнут лимит разбиений.";
result.value = prev;
result.n = n;
result.runge_error = std::numeric_limits<double>::infinity();
return result;
}

namespace {

struct IntervalSegment {
    double from = 0.0;
    double to = 0.0;
    bool limit_at_from = false;
    bool limit_at_to = false;
};

struct IntervalPlan {
    std::string status;
    std::string message;
    std::vector<IntervalSegment> segments;
};

struct LimitResult {
    double value = 0.0;
    int n = 0;
    double runge_error = 0.0;
    std::string status;
};

LimitResult limitAtEndpoint(Integrator &integ, IntegrandFunc f, double c,

```

```

        double other, int side, double eps) {
LimitResult res;
const double span = std::abs(other - c);
const double d0 = span * 1e-2;
const double r = 0.5;
constexpr int kMaxShrinks = 50;
constexpr int kNeedStable = 2;
constexpr double kBigThreshold = 1e15;
const double stableTol = 3.0 * eps;

double prevI = std::numeric_limits<double>::quiet_NaN();
int stableCount = 0;
double lastValue = 0.0;
int lastN = 0;
double lastRunge = 0.0;

for (int kStep = 0; kStep < kMaxShrinks; ++kStep) {
    const double sigma = d0 * std::pow(r, kStep);
    double lo, hi;
    if (side > 0) {
        lo = c + sigma;
        hi = other;
    } else {
        lo = other;
        hi = c - sigma;
    }
    if (!(lo < hi)) {
        res.status = "indeterminate";
        res.value = lastValue;
        res.n = lastN;
        return res;
    }

    IntegrationResult inner = integ.rungeSolve(f, lo, hi);
    if (inner.status == "diverges" || !std::isfinite(inner.value)) {
        res.status = "diverges";
        res.value = std::numeric_limits<double>::quiet_NaN();
        res.n = inner.n;
        return res;
    }
    if (std::abs(inner.value) > kBigThreshold) {
        res.status = "diverges";
        res.value = std::numeric_limits<double>::quiet_NaN();
        res.n = inner.n;
        return res;
    }
    lastValue = inner.value;
    lastN = inner.n;
    lastRunge = inner.runge_error;

    if (std::isfinite(prevI) && inner.status == "ok") {
        const double diff = std::abs(inner.value - prevI);
        if (diff < stableTol) {
            ++stableCount;
            if (stableCount ≥ kNeedStable) {
                res.status = "ok";
                res.value = inner.value;
                res.n = inner.n;
                res.runge_error = lastRunge;
                return res;
            }
        }
    }
}

```

```

    }
    } else {
        stableCount = 0;
    }
} else {
    stableCount = 0;
}
prevI = inner.status == "ok" ? inner.value
                        : std::numeric_limits<double>::quiet_NaN();
}

res.status = "indeterminate";
res.value = lastValue;
res.n = lastN;
res.runge_error = lastRunge;
return res;
}

bool isAntisymmetric(IntegrandFunc f, double c, double d, double tol = 1e-6) {
    const int samples = 10;
    double worst = 0.0;
    for (int i = 0; i < samples; ++i) {
        const double t = static_cast<double>(i) / static_cast<double>(samples - 1);
        const double delta = d * std::pow(1e-3, 1.0 - t) * std::pow(0.99, t);
        const double fp = f(c + delta);
        const double fm = f(c - delta);
        if (!std::isfinite(fp) || !std::isfinite(fm)) {
            return false;
        }
        const double denom = std::max({std::abs(fp), std::abs(fm), 1e-30});
        const double diff = std::abs(fp + fm) / denom;
        if (diff > worst) {
            worst = diff;
        }
    }
    return worst < tol;
}

struct SegmentResult {
    double value = 0.0;
    int n = 0;
    double runge_error = 0.0;
    std::string status;
    std::string message;
};

SegmentResult computeSegmentStandard(Integrator &integ, IntegrandFunc f,
                                     const IntervalSegment &seg, double eps) {
    SegmentResult r;
    if (!seg.limit_at_from && !seg.limit_at_to) {
        IntegrationResult inner = integ.rungeSolve(f, seg.from, seg.to);
        r.value = inner.value;
        r.n = inner.n;
        r.runge_error = inner.runge_error;
        r.status = inner.status;
        r.message = inner.message;
        return r;
    }

    if (seg.limit_at_from && seg.limit_at_to) {

```

```

const double mid = (seg.from + seg.to) / 2.0;
LimitResult lr1 = limitAtEndpoint(integ, f, seg.from, mid, +1, eps);
if (lr1.status != "ok") {
    r.status = lr1.status;
    return r;
}
LimitResult lr2 = limitAtEndpoint(integ, f, seg.to, mid, -1, eps);
if (lr2.status != "ok") {
    r.status = lr2.status;
    return r;
}
r.value = lr1.value + lr2.value;
r.n = lr1.n + lr2.n;
r.runge_error = lr1.runge_error + lr2.runge_error;
r.status = "ok";
return r;
}
if (seg.limit_at_from) {
    LimitResult lr = limitAtEndpoint(integ, f, seg.from, seg.to, +1, eps);
    r.value = lr.value;
    r.n = lr.n;
    r.runge_error = lr.runge_error;
    r.status = lr.status;
    return r;
}
LimitResult lr = limitAtEndpoint(integ, f, seg.to, seg.from, -1, eps);
r.value = lr.value;
r.n = lr.n;
r.runge_error = lr.runge_error;
r.status = lr.status;
return r;
}

IntervalPlan prepareIntervals(double a, double b,
                               const std::vector<double> &discontinuities) {
    IntervalPlan plan;
    plan.status = "ok";

    std::vector<double> innerPoints;
    bool startIsSingular = false;
    bool endIsSingular = false;

    for (double c : discontinuities) {
        if (std::abs(c - a) < 1e-15) {
            startIsSingular = true;
        } else if (std::abs(c - b) < 1e-15) {
            endIsSingular = true;
        } else if (c > a && c < b) {
            innerPoints.push_back(c);
        }
    }
    std::sort(innerPoints.begin(), innerPoints.end());

    if (innerPoints.empty()) {
        IntervalSegment seg;
        seg.from = a;
        seg.to = b;
        seg.limit_at_from = startIsSingular;
        seg.limit_at_to = endIsSingular;
        plan.segments.push_back(seg);
    }
}

```

```

    return plan;
}

double prev = a;
bool prevSingular = startIsSingular;
for (double c : innerPoints) {
    IntervalSegment seg;
    seg.from = prev;
    seg.to = c;
    seg.limit_at_from = prevSingular;
    seg.limit_at_to = true;
    plan.segments.push_back(seg);
    prev = c;
    prevSingular = true;
}
IntervalSegment tail;
tail.from = prev;
tail.to = b;
tail.limit_at_from = prevSingular;
tail.limit_at_to = endIsSingular;
plan.segments.push_back(tail);
return plan;
}

struct PvOutcome {
    bool success = false;
    bool usedPV = false;
    double value = 0.0;
    int n = 0;
    double runge_error = 0.0;
    std::string message;
};

PvOutcome tryPrincipalValue(Integrator &integ, IntegrandFunc f,
                           const IntervalPlan &plan, double eps) {

    PvOutcome out;
    double acc = 0.0;
    int nAcc = 0;
    double rg = 0.0;
    bool usedPV = false;

    size_t i = 0;
    while (i < plan.segments.size()) {
        const IntervalSegment &seg = plan.segments[i];

        if (!seg.limit_at_from && !seg.limit_at_to) {
            IntegrationResult inner = integ.rungeSolve(f, seg.from, seg.to);
            if (inner.status != "ok") {
                out.message = "Подотрезок не сошёлся.";
                return out;
            }
            acc += inner.value;
            nAcc += inner.n;
            rg += inner.runge_error;
            ++i;
            continue;
        }

        bool handled = false;
        if (i + 1 < plan.segments.size()) {

```

```

const IntervalSegment &nxt = plan.segments[i + 1];
if (seg.limit_at_to && nxt.limit_at_from &&
    std::abs(seg.to - nxt.from) < 1e-15) {
    const double c = seg.to;
    const double leftLen = c - seg.from;
    const double rightLen = nxt.to - c;
    const double d = std::min(leftLen, rightLen);

    if (d > 0.0 && isAntisymmetric(f, c, d)) {
        double leftTailVal = 0.0;
        int leftTailN = 0;
        double leftTailRunge = 0.0;
        if (leftLen > d + 1e-15) {
            if (seg.limit_at_from) {
                LimitResult lr =
                    limitAtEndpoint(integ, f, seg.from, c - d, +1, eps);
                if (lr.status != "ok") {
                    out.message = "Хвостовой подотрезок расходится.";
                    return out;
                }
                leftTailVal = lr.value;
                leftTailN = lr.n;
                leftTailRunge = lr.runge_error;
            } else {
                IntegrationResult inner = integ.rungeSolve(f, seg.from, c - d);
                if (inner.status != "ok") {
                    out.message = "Хвостовой подотрезок не сошёлся.";
                    return out;
                }
                leftTailVal = inner.value;
                leftTailN = inner.n;
                leftTailRunge = inner.runge_error;
            }
        }

        double rightTailVal = 0.0;
        int rightTailN = 0;
        double rightTailRunge = 0.0;
        if (rightLen > d + 1e-15) {
            if (nxt.limit_at_to) {
                LimitResult lr =
                    limitAtEndpoint(integ, f, nxt.to, c + d, -1, eps);
                if (lr.status != "ok") {
                    out.message = "Хвостовой подотрезок расходится.";
                    return out;
                }
                rightTailVal = lr.value;
                rightTailN = lr.n;
                rightTailRunge = lr.runge_error;
            } else {
                IntegrationResult inner = integ.rungeSolve(f, c + d, nxt.to);
                if (inner.status != "ok") {
                    out.message = "Хвостовой подотрезок не сошёлся.";
                    return out;
                }
                rightTailVal = inner.value;
                rightTailN = inner.n;
                rightTailRunge = inner.runge_error;
            }
        }
    }
}

```

```

        acc += leftTailVal + rightTailVal;
        nAcc += leftTailN + rightTailN;
        rg += leftTailRunge + rightTailRunge;
        usedPV = true;
        handled = true;
        i += 2;
    }
}

if (handled) {
    continue;
}

SegmentResult sr = computeSegmentStandard(integ, f, seg, eps);
if (sr.status != "ok") {
    out.message = "Несобственный подотрезок расходится.";
    return out;
}
acc += sr.value;
nAcc += sr.n;
rg += sr.runge_error;
++i;
}

out.success = true;
out.usedPV = usedPV;
out.value = acc;
out.n = nAcc;
out.runge_error = rg;
return out;
}

}

IntegrationResult Integrator::integrate(IntegrandFunc f, double a, double b,
                                         const std::vector<double> &discontinuities) {
    IntegrationResult out;
    if (!(a < b)) {
        out.status = "error";
        out.message = "Левый предел должен быть меньше правого.";
        out.value = std::numeric_limits<double>::quiet_NaN();
        return out;
    }
    if (!(EPS > 0.0) || !std::isfinite(EPS)) {
        out.status = "error";
        out.message = "Некорректная точность ε.";
        out.value = std::numeric_limits<double>::quiet_NaN();
        return out;
    }

    IntervalPlan plan = prepareIntervals(a, b, discontinuities);
    const bool improper = plan.segments.size() > 1 ||
                         plan.segments.front().limit_at_from ||
                         plan.segments.front().limit_at_to;

    double total = 0.0;
    int totalN = 0;
    double totalRunge = 0.0;

```

```

std::string failedStatus;

for (const IntervalSegment &seg : plan.segments) {
    SegmentResult sr = computeSegmentStandard(*this, f, seg, EPS);
    if (sr.status == "max_iter") {
        out.status = "max_iter";
        out.message = "Не удалось достичь точности за отведённые разбиения.";
        out.value = sr.value;
        out.n = sr.n;
        out.runge_error = sr.runge_error;
        return out;
    }
    if (sr.status != "ok") {
        failedStatus = sr.status;
        total += std::isfinite(sr.value) ? sr.value : 0.0;
        totalN += sr.n;
        totalRunge += sr.runge_error;
        break;
    }
    total += sr.value;
    totalN += sr.n;
    totalRunge += sr.runge_error;
}

if (failedStatus.empty()) {
    out.status = "ok";
    out.value = total;
    out.n = totalN;
    out.runge_error = totalRunge;
    out.message = improper ? "Несобственный интеграл, сходится."
                          : "Интеграл вычислен.";
    return out;
}

PvOutcome pv = tryPrincipalValue(*this, f, plan, EPS);
if (pv.success && pv.usedPV) {
    out.status = "ok_principal_value";
    out.value = pv.value;
    out.n = pv.n;
    out.runge_error = pv.runge_error;
    out.message = "Несобственный интеграл, главное значение Коши.";
    return out;
}

if (failedStatus == "indeterminate") {
    out.status = "indeterminate";
    out.value = total;
    out.n = totalN;
    out.runge_error = totalRunge;
    out.message =
        "Численно не удалось установить сходимость предела (метод плохо "
        "приспособлен к особенности – попробуйте другой).";
    return out;
}

out.status = "diverges";
out.value = std::numeric_limits<double>::quiet_NaN();
out.message = "Интеграл расходится.";
return out;
}

```

Полный исходный код проекта (Qt6/QML/C++ десктоп-приложение) доступен в репозитории:

`git.oriptal.dev/cadmin/calc-math-lab2`

3 Выводы

В ходе лабораторной работы изучены и программно реализованы численные методы вычисления определённых интегралов: левые, правые и средние прямоугольники, трапеции и Симпсон. Для варианта №5 точное значение $I = -160$ получено по формуле Ньютона–Лейбница; при $n = 6$ общая формула Ньютона–Котеса даёт его же ровно, при $n = 10$ Симпсон также даёт точное значение, средние прямоугольники — -159.86 (отн. погрешность 0.0875%), трапеции — -160.28 (0.175% , ровно вдвое больше средних прямоугольников). Несобственные интегралы 2-го рода обрабатываются корректно: сходимость определяется итеративным сужением к точке разрыва, расходимость — отсутствием стабилизации или превышением порога 10^{15} . В качестве расширения реализован fallback на главное значение Коши для антисимметричных особенностей, что позволяет получить корректный ответ $\approx \ln 2$ для интеграла $\int_{-1}^2 \frac{dx}{x}$.

Стоит явно отметить ограничение реализованной схемы проверки сходимости несобственных интегралов. Критерий $|I_k - I_{k-1}| < 3\varepsilon$ для двух подряд k — это инженерное приближение классического критерия Коши: формально для сходимости требуется, чтобы все пары (I_m, I_n) при $m, n \geq N$ были близки, а мы проверяем только две соседних подряд. Для типичных интегрантов с монотонной или почти монотонной структурой разностей этого достаточно, но на сильно осциллирующих сингулярностях вида $\sin(1/x)/x$ две соседние разности могут случайно совпасть из-за фазы осцилляции, и метод может ошибочно отметить интеграл как сошедшийся. Аналогично, очень медленные расходимости вида $1/(x \ln x)$ за конечное число итераций могут стать неотличимыми от ещё более

медленных сходимостей вида $1/(x \ln^2 x)$. На штатных функциях задания (полиномы, $1/x$, $1/\sqrt{x}$) этот класс ошибок не проявляется.